

INTRODUZIONE

L'obiettivo di questo progetto è stato quello di modellare una mano robotica che emulasse il comportamento di presa di una mano umana con la finalità di essere usata come protesi.

MODELLAZIONE

Le dita sono state modellate come:

- 4 dita come manipolatori planari a tre braccia distanziati di una distanza di 2 cm l'una dall'altra;
- il pollice come un manipolatore antropomorfo, avente gli ultimo due giunti con asse di rotazione, in posizione di riposo, paralleli a quelli delle 4 dita e il giunto alla base ruotato di 90 gradi rispetto a gli altri due per simulare le caratteristiche di opponibilità di un vero pollice.

```
L_ind = [ Link([0 0 0.045 0]), Link([0 0 0.025 0]), Link([0 0 0.020 0]) ];
L_med = [ Link([0 0 0.050 0]), Link([0 0 0.030 0]), Link([0 0 0.025 0]) ];
L_anu = [ Link([0 0 0.045 0]), Link([0 0 0.025 0]), Link([0 0 0.020 0]) ];
L_mig = [ Link([0 0 0.035 0]), Link([0 0 0.020 0]), Link([0 0 0.015 0]) ];
L_pol = [ Link([0 0 0 pi/2]), Link([0 0 0.040 0]), Link([0 0 0.030 0]) ];

indice = SerialLink(L_ind, 'name', 'Indice');
medio  = SerialLink(L_med, 'name', 'Medio');
anulare = SerialLink(L_anu, 'name', 'Anulare');
mignolo = SerialLink(L_mig, 'name', 'Mignolo');
pollice = SerialLink(L_pol, 'name', 'Pollice');

L_palmo = 0.05;
indice.base = transl(L_palmo, 0, 0.02);
medio.base  = transl(L_palmo, 0, 0);
anulare.base = transl(L_palmo, 0, -0.02);
mignolo.base = transl(L_palmo, 0, -0.04);
pollice.base = transl(0.02, 0, 0.04) * troty(-pi/2);
```

Sono stati quindi definiti i vari bracci di ogni manipolatore come un vettore di `Link`, variabile definita dai parametri D-H, e poi essere collegati in una variabile `SerialLink`. Tali bracci saranno poi rappresentati rispetto a una terna base con la quale si rappresenteranno distanze e rotazione tra i vari manipolatori. Il pollice è stato ruotato di -90 gradi in modo permettere alla rotazione del giunto alla base di esso di non ruotare su se stesso.

```
r_cil = 0.015;
pos_cil = [0.03, 0.02, 0];
h_cil = 0.12;
```

Per simulare la presa è stato creato un oggetto di riferimento, ossia un cilindro avente come parametri raggio, altezza e posizione del centro nelle 3 coordinate spaziali.

PUNTI DI CONTATTO

Al fine di emulare il comportamento di un prototipo reale provvisto di sensori di distanza tra le punta delle dita e il punto di contatto sono stati calcolati i punti di contatto per le 4 dita tramite una funzione di ottimizzazione avente come funzione obiettivo la distanza euclidea tra le punte e il cilindro

```
%CALCOLO PUNTI DI CONTATTO 4 DITA
f_vincolo = @(s) [s, 1.2*s, 0.8*s];
obiettivo_dita = @(q, robot, pos_c, r_c) abs(norm(robot.fkine(q).t(1:2) -
pos_c(1:2)') - r_c);

dita_lunghe = {indice, medio, anulare, mignolo};
nomi_lunghe = {'Indice', 'Medio', 'Anulare', 'Mignolo'};
q_contatto = cell(1, 5);

for i = 1:4
    s_opt = fminbnd(@(s) obiettivo_dita(f_vincolo(s), dita_lunghe{i}, pos_cil,
r_cil), 0, pi/2);
    q_contatto{i} = f_vincolo(s_opt);
    fprintf('%s -> Target trovato con s = %.2f rad\n', nomi_lunghe{i}, s_opt);
end
```

Per il pollice invece è stato calcolato appositamente tramite cinematica inversa attraverso il metodo `ikine`. Le differenze tra questi due approcci è nell'utilizzo di `f_vincolo = @(s) [s, 1.2*s, 0.8*s]`; per il calcolo delle 4 dita. Inizialmente il sistema era stato pensato per essere sottoattuato, dove ogni dito veniva controllato tramite un unico cavo, e tutti i cavi venivano avvolti da un unico motore. Era presente solo un secondo motore per permettere alla base del pollice di ruotare autonomamente. A causa di difficoltà nel corretto posizionamento del pollice si è tornati a un modello non sotto attuato, mantenendo tale vincolo per rappresentare la chiusura naturale di una mano.

```
T_pol_target = transl(pos_cil(1), pos_cil(2) + r_cil, 0.04);
q_contatto{5} = pollice.ikine(T_pol_target, 'mask', [1 1 1 0 0 0]);
q_p = q_contatto{5};
```

TRAJETTORIA

```
dt = 0.001;
t_traj = 0:dt:1.5; % traiettoria di 1.5 secondi
Q_des = cell(1, 5);
Qd_des = cell(1, 5);

for i = 1:5
    target = q_contatto{i}(:)';
    [Q_des{i}, Qd_des{i}, ~] = jtraj([0 0 0], target, t_traj);
end
```

Si definisce un passo `dt` e un tempo finale sul quale verranno calcolati i punti della traiettoria tramite la funzione `jtraj`. Questa funzione utilizza un polinomio di 5° in modo da avere velocità e accelerazioni continue e ottenere quindi un movimento fluido.

DINAMICA E CONTROLLO

Vengono definite due funzioni:

```
function robot = add_dynamic_props(robot)
    rho = 4000; % Densità materiale
    for j = 1:robot.n
        L = robot.links(j);
        a = L.a; % Lunghezza link
        if a == 0, a = 0.001; end % Evita divisione zero
        r_p = 0.005; % Raggio sezione
        volume = pi * r_p^2 * a; % Volume cilindro
        massa = rho * volume; % Massa calcolata
        L.m = massa; % Assegnazione massa
        L.r = [a/2, 0, 0]; % Centro di massa

        % Inerzie principali (Formula cilindro)
        Ixx = 0.5 * massa * r_p^2; % Inerzia rotazione asse
        Iyy = (1/12) * massa * (3*r_p^2 + a^2); % Inerzia flessione Y
        Izz = Iyy; % Inerzia flessione Z
        L.I = [Ixx, Iyy, Izz, 0, 0, 0]; % Tensore d'inerzia

        % Parametri attuatore e attriti
        L.G = 1; % Rapporto riduzione
        L.Jm = 1e-4; % Inerzia rotore
        L.B = 1e-4; % Attrito viscoso
        L.Tc = [1e-4, 1e-4]; % Attrito Coulomb
    end
    robot = SerialLink(robot); % Aggiornamento oggetto
end
```

La prima funzione sopra mostrata serve per assegnare le proprietà fisiche ai manipolatori. Si è scelto di approssimare i segmenti delle dita come dei cilindri pieni omogenei. In particolare si ha:

1. Densità ($\rho = 4000 \text{ kg/m}^3$): per fare un confronto l'alluminio è circa 2700 kg/m^3 , mentre il titanio è 4500 kg/m^3 . Hai scelto un materiale metallico leggero o una plastica molto densa/caricata;
2. Raggio della sezione ($r_p = 0.005 \text{ m}$);
3. Massa dipendente seguendo la geometria del cilindro tramite densità, lunghezza e raggio;
4. Baricentro impostato a metà lunghezza delle dita;
5. Momenti di inerzia di un cilindro:

- I_{xx} (Momento d'inerzia polare): Resistenza alla rotazione attorno all'asse del dito. $I_{xx} = \frac{1}{2} m r^2$

- I_{yy} e I_{zz} (Momenti trasversali): Resistenza al movimento di flessione (quello principale del dito $I_{yy} = I_{zz} = \frac{1}{12} m (3r^2 + a^2)$)

6. I prodotti d'inerzia sono stati posti a zero, assumendo che gli assi di simmetria del cilindro coincidano perfettamente con gli assi del sistema di riferimento dei giunti;
7. Rapporto di riduzione unitario per presa diretta;
8. Inerzia del rotore, attrito viscoso e coulomb sono valori inseriti per rendere più realistica la simulazione.

```
function tau = control_law(robot, t, q, qd, t_traj, Q_des, Qd_des, Kp, Kd)
    q = q(:)';
    qd = qd(:)';

    % Interpolazione per trovare il punto desiderato all'istante t
    if t <= t_traj(end)
        q_d = interp1(t_traj, Q_des, t, 'spline');
        qd_d = interp1(t_traj, Qd_des, t, 'spline');
    else
        q_d = Q_des(end, :);
        qd_d = [0, 0, 0];
    end

    % Calcolo Errore
    e = q_d - q;
    ed = qd_d - qd;

    % Compensazione di Gravità + Controllo PD
    G = robot.gravload(q);
    tau = (Kp * e + Kd * ed) + G;
end
```

La funzione `control_law` implementa la legge di controllo incaricato di azionare le dita lungo la traiettoria pianificata. L'algoritmo calcola l'errore di posizione e di velocità e, moltiplicati per i coefficienti `Kp` e `Kd`, rappresentano l'azione proporzionale e derivativa del controllore, alla quale si somma la componente di gravità in modo da compensarla. Poiché la traiettoria desiderata è stata generata in modo discreto (una sequenza di punti), ma il solutore ODE interno alla funzione `fdyn()` opera nel tempo continuo, si è reso necessario l'utilizzo della funzione `interp1`. L'interpolazione di tipo Cubic Spline garantisce che, per ogni istante t richiesto dal solutore, i riferimenti di posizione (q_d) e velocità ($\dot{q}_d$) siano non solo disponibili, ma anche derivabili e fluidi. Questo evita discontinuità matematiche (gradini) che causerebbero picchi di coppia fisicamente irrealizzabili e instabilità numerica nella simulazione.

```
for i = 1:5
    fprintf('Simulazione dito %d in corso...\n', i);
    robot = dita{i};
    ctrl_handle = @(robot, t, q, qd) control_law(robot, t, q, qd, t_traj,
        Q_des{i}, Qd_des{i}, Kp, Kd);
    [t_out, q_out, qd_out] = robot.fdyn(T_sim, ctrl_handle, q0, qd0);
end
```

```

t_out = t_out(:);
[t_out_uniq, idx_uniq] = unique(t_out);
q_out_uniq = q_out(idx_uniq, :);
qd_out_uniq = qd_out(idx_uniq, :);

qddot_raw = zeros(length(t_out_uniq), 3);

for k = 1:length(t_out_uniq)
    coppia_raw = control_law(robot, t_out_uniq(k), q_out_uniq(k,:),
qd_out_uniq(k,:), ...
                                t_traj, Q_des{i}, Qd_des{i}, Kp, Kd);

    qddot_raw(k,:) = robot.accel(q_out_uniq(k,:), qd_out_uniq(k,:),
coppia_raw)';

end

q_hist{i} = interp1(t_out_uniq, q_out_uniq, t, 'pchip', 'extrap');
qd_hist{i} = interp1(t_out_uniq, qd_out_uniq, t, 'pchip', 'extrap');
qddot_hist{i} = interp1(t_out_uniq, qddot_raw, t, 'pchip', 'extrap');

end

```

Per ognuna delle dita ne si calcola la dinamica tramite la funzione `fdyn`, la quale effettua il calcolo della dinamica e, integrando, restituisce posizione e velocità del manipolatore. In modo da calcolarci l'accelerazione, effettuiamo il calcolo della coppia generata dalla nostra legge di controllo per poi essere usata dal metodo `accel()`. I valori di posizione, velocità e accelerazione vengono infine interpolati per ottenere dati associati agli stessi istanti di tempo e quindi comparabili.

SIMULINK

Il sistema in retroazione così come descritto fino ora è visualizzabile tramite il seguente schema simulink